

Agent Idol – Tech Stack

Companion to the v2 design handover. Covers the web build only.

1. What the design forces

Four constraints drive every choice below.

1. **A 30-minute timed run scored on one number.** Timing has to be exact and tamper-resistant enough to trust. This is the single biggest architectural driver.
2. **Escalating click pressure.** Input latency has to stay low with dozens of live sprites on screen. Dropped or delayed clicks are a direct insult to the core fantasy.
3. **A pixel grid that must not smear.** Fixed stage, integer scaling, no subpixel anything.
4. **No derived numbers on screen.** Almost everything the player reads is motion, so

the animation layer is load-bearing rather than decorative.

2. Core stack

Layer	Choice
Language	TypeScript, strict
Build	Vite
Framework	None
Rendering	DOM + CSS
Motion	anime.js v4
Audio	Native Web Audio, thin wrapper
Persistence	<code>localStorage</code> , versioned JSON
Tests	Vitest
Deploy	Static bundle — Cloudflare Pages, Netlify, or itch.io

Why no framework

The game floor updates continuously: every progress bar, every payout counter, every credit tick, sixty times a second. A component framework wants to own that update path and will re-render its way through your frame budget for no benefit. React's value is managing complex conditional UI trees, and this UI is a fixed grid of four pods that never changes shape.

Vanilla TypeScript with a small hand-rolled renderer is less code here, not more.

If the shop, run-summary and reputation screens grow into something genuinely tree-like, they can be a separate framework-rendered layer that mounts over a paused floor. Don't do that pre-emptively.

Why anime.js v4 specifically

v4 is ES-module and tree-shakeable, around 10KB gzipped for the full library, with named imports (`animate`, `createTimeline`,

`stagger`, `createDraggable`, `utils`) rather than the v3 global. Import only what you use.

Two v4 features matter directly:

- `createTimeline()` for orchestrated sequences — the repossession stagger, the endgame transition, the run-end tally.
- `createDraggable()` — worth evaluating for the amber-drag gesture before hand-rolling pointer events. It handles the pickup ghost and release spring you'd otherwise write yourself.

3. The one architectural rule

Simulation and rendering are separate modules, and the simulation never touches the DOM.

```
src/  
  sim/           pure state + rules.  
No DOM, no anime.js, no window.  
  state.ts  
  tick.ts  
  rng.ts
```

```
economy.ts
projects.ts
render/      reads sim state,
writes to DOM
  floor.ts
  pod.ts
  agent.ts
  topbar.ts
fx/          anime.js only.
Discrete events, never continuous
state.
ui/          shop, run summary,
reputation, title
main.ts      loop, input, wiring
```

This buys three things that are hard to retrofit:

Deterministic runs. Seeded RNG plus a fixed timestep means a run is reproducible from (seed, input log). That's the foundation for replays, for verified leaderboard scores, and for debugging a run someone reports.

Headless balance testing. The economy has a lot of interacting knobs — decay rates, difficulty penalties, token prices falling with deliveries, block pricing, the escalation curve. You can run ten thousand simulated 30-minute runs under

scripted strategies in a Vitest suite and read the score distribution, instead of playing the game five hundred times. For a score-attack game this is the difference between tuned and guessed.

A sane pause. Freezing a pure state object on blur is trivial. Freezing a DOM-entangled one is not.

The loop

Fixed-timestep simulation, decoupled render:

```
const STEP = 1000 / 30;      // sim
                               ticks at 30Hz; render at display rate
let acc = 0, last =
performance.now();

function frame(now: number) {
  let delta = now - last;
  last = now;
  delta = Math.min(delta, 250); //
  clamp — no catch-up burst after a
  stall
  acc += delta;
  while (acc >= STEP) { tick(state,
STEP); acc -= STEP; }
```

```
render(state, acc / STEP);    //  
alpha for interpolation  
requestAnimationFrame(frame);  
}
```

The delta clamp matters. Without it, a tab that stalls for four seconds resolves twelve agent runs in one frame and drains a credit block instantly.

Pause on blur

Listen on `visibilitychange` and `blur`. On hide: stop the loop, freeze the run clock, note the timestamp. On show: reset `last` to now before resuming, so no accumulated delta leaks in. The run clock is wall-clock-independent — it only advances while ticking.

4. Rendering notes

Transforms only. The current mockup animates progress bars with `width: n%`, which triggers layout on every frame for every bar. Switch to `transform: scaleX()` with `transform-origin: left`. Same look,

composited, no layout. Same for the credit bar and the segmented project bar.

One DOM node per agent, reused. Agents move between pods by reparenting, not by teardown and rebuild. State changes are class swaps: `.agent.is-running`, `.is-blocked`, `.is-idle`, `.is-ok`. The four state colours already live in CSS custom properties.

Sprites: pre-render once. Don't run the 16×16 pixel-plot loop per canvas per state as the mockup does. At boot, draw each `class × state` combination once into an offscreen canvas, export to a data URL, and use plain `<img>` elements with `image-rendering: pixelated`. Twelve images total, zero per-frame canvas work, and `<img>` is cheaper to reparent and transform than `<canvas>`.

Stage scaling. One `#stage` element at exactly 1280×720, `transform: scale(n) translate(...)` where `n` is `Math.floor(min(vw/1280, vh/720))` — integer only, letterboxed with the background colour. Never fractional, or the pixel grid smears.

Self-host the fonts. Silkscreen, JetBrains Mono and Inter as local `woff2` with `font-display: block`. A Google Fonts round-trip means the first frame renders in a fallback face and the pixel UI visibly reflows.

5. Where anime.js is allowed

This is a boundary worth policing. Two independent clocks driving the same values produces bugs that are miserable to trace.

anime.js owns discrete, one-shot, cosmetic events:

- A slice landing in the segmented progress bar
- The `+15%` pop on a successful run
- Sprite walks between pods on reassignment
- The blocked-agent bubble bob (or leave this as pure CSS `@keyframes` — it's cheaper)
- Repossession, as a `createTimeline()` stagger

- The endgame sequence at T-2:00 – timer bob, floor darkening, panel edges tightening
- Screen transitions: run start, run end, reputation spend

The simulation owns everything continuous:

- The payout countdown
- Credit bar drain
- Agent run progress
- The run clock

Rule of thumb: if pausing the game must freeze it, the sim owns it.

Respect prefers-reduced-motion. The mockups already kill animation under it, but note the design leans on motion to communicate rate. Under reduced motion, the payout counter still needs to update and the credit bar still needs to drain – cut the decorative layer (bob, walks, pops, stagger), keep the informational layer.

6. Input

Click. Direct listeners on blocked agents. Use `pointerdown`, not `click`, for the retry — roughly 50–80ms of perceived latency saved per interaction, which is worth having in a game whose late run is nothing but clicking.

Drag. Evaluate `createDraggable()` from anime.js v4 first. If it doesn't fit, hand-roll with Pointer Events (`setPointerCapture`) — never the HTML5 drag-and-drop API, which has unreliable custom drag images, poor cursor control and inconsistent cross-browser behaviour.

Drop targets are already dashed boxes in the visual language. Highlight on hover by brightening the dash, not by adding a new colour — the five-colour rule holds.

Keyboard. Not a design goal, but the quality floor asks for visible focus. Tab through blocked agents, Enter to retry, is cheap and makes the endgame less punishing on wrists.

7. Persistence

Only reputation and unlocks persist. A run in progress is not saved — the clock is the point.

```
type Save = {  
  v: 1; //  
  schema version, always check on load  
  reputation: number;  
  unlocked: string[]; //  
  classes, upgrades, block sizes  
  roster: { class: string }[];  
  best: { score: number; seed: string  
}[];  
};
```

`localStorage` is sufficient — this is a few KB. IndexedDB is unnecessary complexity unless you later store full replay logs.

Guard every load: try/catch the parse, check `v`, fall back to a fresh save rather than crashing on corrupt data.

Note for artifact prototypes: browser storage APIs are unavailable inside Claude artifacts. Any prototype built there is in-memory only, so reputation won't survive a reload. This is a sandbox limitation, not a product constraint.

8. Audio

A clicker lives or dies on click feedback. Native Web Audio with a small wrapper is enough — you need maybe eight one-shots (retry click, run complete, project delivered, credits bought, credits empty, repossession, endgame warning, buzzer) and one ambient loop.

Don't pull in Howler for that. Do decode buffers at boot, and do gate everything behind the first user gesture or Chrome blocks the context.

Worth considering: a subtle rising pitch or tempo on the ambient bed as the clock runs down. It communicates urgency without breaking the no-red-clock rule.

9. What not to use

Avoid	Why
React / Vue / Svelte	Fixed UI shape, continuous per-frame updates — the

Avoid	Why
	framework is pure overhead here
Phaser / PixiJS	The game is panels, bars and text. WebGL buys nothing and costs the crisp DOM text
Redux / Zustand / signals	One plain mutable state object, one owner, one tick function
Tailwind	The design is a small closed token system already expressed as CSS custom properties
HTML5 drag-and-drop API	Unreliable drag images and cursor behaviour; Pointer Events are better in every respect
<code>setInterval</code> for game logic	Drifts, and doesn't stop cleanly on blur
Animating <code>width</code> / <code>left</code> /	Layout thrash. Transforms only

Avoid	Why
top	
Google Fonts CDN	First-frame reflow on a pixel-grid UI

10. Build order

1. **Sim core with no renderer.** State, tick, seeded RNG, one project, one agent. Assert against it in Vitest.
2. **Static floor.** Port the v9 mockup markup, strip the fake data, bind to sim state. No animation yet.
3. **The loop.** Fixed timestep, blur pause, delta clamp, run clock on screen.
4. **The two gestures.** Click-to-retry, drag-to-assign.
5. **Economy.** Credits, blocks, the dial, falling token prices, debt slope.
6. **Headless balance harness.** Scripted strategies × thousands of runs, before hand-tuning anything.

7. **anime.js pass.** All the juice at once, so it can be judged as a whole.

8. **Meta layer.** Run summary, reputation, unlock catalogue, roster.

9. **Run 1 tutorial ramp.** Last, because it's a special case of everything else.

Steps 1–4 are a playable thing you can feel. If 30 minutes turns out to be the wrong clock, you'll know by step 4 and nothing after it is wasted.